

**«Санкт-Петербургский государственный электротехнический университет  
«ЛЭТИ» им. В.И.Ульянова (Ленина)»  
(СПбГЭТУ «ЛЭТИ»)**

**ОТЧЁТ  
ПО УЧЕБНОЙ ПРАКТИКЕ**

**Тема: Генетические алгоритмы.**

Студенты г. 4341

Руководитель

К.А. Берсенев

А.С. Большакова

Д.Р. Валитов

Т.Р. Жангиров

Санкт-Петербург

2026

## ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студенты К.А. Берсенов, А.С. Большакова, Д.Р. Валитов

Группа: 4341

Тема практики: Генетические алгоритмы

Задание на практику:

Задача о назначениях

Пусть имеется  $N$  работ и  $N$  кандидатов на выполнение этих работ, причем назначение  $j$ -й работы  $i$ -му кандидату требует затрат  $c_{ij} > 0$ . Необходимо назначить каждому кандидату по работе, чтобы минимизировать суммарные затраты. Причем каждый кандидат может быть назначен на одну работу, а каждая работа может выполняться только одним кандидатом.

Сроки прохождения практики: 06.07.2026-18.07.2026

Дата сдачи отчета: 16.07.2026

Дата защиты отчета: 17.07.2026

Студент

\_\_\_\_\_

К.А. Берсенов  
А.С. Большакова  
Д.Р. Валитов

Руководитель

\_\_\_\_\_

Т.Р. Жангиров

## АННОТАЦИЯ

Разработана программа для решения задачи о назначениях: имеются  $N$  работ и  $N$  кандидатов, назначение  $j$ -й работы  $i$ -му кандидату требует затрат  $c_{ij} > 0$ . Необходимо минимизировать суммарные затраты при условии, что каждый кандидат выполняет одну работу, а каждая работа — только одним кандидатом.

В основе решения — генетический алгоритм с хромосомой-перестановкой. Реализованы формирование популяции, анализ решений целевой функцией, скрещивание и мутация с заданной вероятностью. Программа написана на Python и снабжена графическим интерфейсом, позволяющим задавать размерность, генерировать данные, загружать их из файла и управлять параметрами ГА (размер популяции, число поколений, вероятности мутации и скрещивания).

Предусмотрена визуализация хода поиска: для каждой популяции выводится текущее и глобальное лучшее решение и значение целевой функции, строится график целевой функции, доступен пошаговый режим с возможностью возврата на несколько шагов назад и переход к финальному результату.

## SUMMARY

A program has been developed to solve the assignment problem: there are  $N$  jobs and  $N$  candidates, and assigning the  $j$ -th job to the  $i$ -th candidate incurs a cost  $c_{ij} > 0$ . The goal is to minimize the total cost under the constraint that each candidate is assigned to exactly one job and each job is performed by exactly one candidate.

The solution is based on a genetic algorithm with permutation-based chromosome encoding. The implementation includes population initialization, solution evaluation and selection for subsequent operations, crossover and mutation with specified probabilities. The program is written in Python and features a graphical user interface that allows the user to set the problem size, generate random data, load data from a file, and adjust GA parameters (population size, number of generations, crossover and mutation probabilities).

Visualization of the search process is provided: for each population, the current best solution and the global best solution along with their objective function values are displayed, a convergence graph of the fitness function is plotted incrementally, and a step-by-step mode is available with the ability to go back several steps as well as to jump directly to the final result.

## СОДЕРЖАНИЕ

|                                                         |    |
|---------------------------------------------------------|----|
| ВВЕДЕНИЕ .....                                          | 6  |
| 1 ПОСТАНОВКА ЗАДАЧИ .....                               | 7  |
| 1.1 Предметная область .....                            | 7  |
| 1.2 Задачи практики .....                               | 7  |
| 1.3 Этапы работы.....                                   | 8  |
| 1.4 Требования к реализации .....                       | 8  |
| 2 РЕЗУЛЬТАТЫ РАБОТЫ.....                                | 10 |
| 2.1. Организация работы и распределение подзадач.....   | 10 |
| 2.2. Структура данных и хранение состояния.....         | 10 |
| 2.3. Генетический алгоритм. Операторы и параметры. .... | 11 |
| 2.4. Идея синхронизации GUI и ГА.....                   | 12 |
| 2.5. Реализация GUI .....                               | 13 |
| 2.6. Реализация генетического алгоритма .....           | 16 |
| 2.7. Реализация контроллера .....                       | 18 |
| 2.8. Реализация навигации.....                          | 20 |
| 3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ .....                         | 21 |
| 4 ОТЗЫВ РУКОВОДИТЕЛЯ .....                              | 22 |
| ЗАКЛЮЧЕНИЕ .....                                        | 23 |
| СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ .....                  | 24 |
| ПРИЛОЖЕНИЕ А .....                                      | 25 |

## **ВВЕДЕНИЕ**

Целью практики является разработка программы для решения задачи о назначениях на основе генетического алгоритма, обеспечивающего визуализацию процесса поиска и интерактивное управление ходом вычислений. Для достижения поставленной цели необходимо реализовать три функциональных модуля — графический интерфейс пользователя, ядро генетического алгоритма и контроллер, отвечающий за синхронизацию между ними, — которые будут разрабатываться параллельно тремя участниками бригады. В ходе выполнения практики планируется изучить принципы построения генетических алгоритмов, методы отбора, скрещивания и мутации для задач с перестановками, а также освоить инструменты создания оконных приложений на Python с библиотекой Tkinter и встраивания графиков Matplotlib. Итоговый продукт должен позволять задавать входные данные, настраивать параметры алгоритма, пошагово наблюдать за эволюцией популяции, отслеживать изменение целевой функции и возвращаться к предыдущим состояниям. Более детальное описание каждого модуля, их взаимодействия и реализованных операторов представлено в соответствующих разделах отчёта.

# **1 ПОСТАНОВКА ЗАДАЧИ**

## **1.1 Предметная область**

Генетические алгоритмы применяются в основном в задачах дискретной и комбинаторной оптимизации, где классические точные методы становятся слишком трудоёмкими из-за резкого роста числа возможных вариантов при увеличении размерности задачи. Также применяются в ситуациях, где нужно одновременно учитывать несколько целей (например, минимизировать затраты и максимизировать качество), и когда целевая функция не имеет простой математической формулы или содержит много локальных максимумов и минимумов. Задача о назначении кандидатов на работы относится именно к комбинаторному типу и является типичной для таких прикладных областей, как кадровое планирование, распределение производственных ресурсов и оперативное управление, где необходимо быстро находить приемлемое решение без потери качества.

## **1.2 Задачи практики:**

1. Изучить теоретические основы задачи о назначениях, формальную постановку и существующие методы решения.
2. Изучить принципы работы генетических алгоритмов: формирование популяции решений, операторы отбора, скрещивания, мутации.
3. Разбить задание на подзадачи и этапы, распределить роли в команде.
4. Разработать и реализовать на языке Python ядро генетического алгоритма для задачи о назначениях (инициализация популяции, анализ целевой функцией, скрещивание решений, мутация, анализ).
5. Разработать графический интерфейс пользователя (GUI) на базе Tkinter, обеспечивающий ввод данных, настройку параметров алгоритма и визуализацию процесса поиска.
6. Обеспечить пошаговый режим работы алгоритма с возможностью возврата на несколько шагов назад и перехода к финальному решению.

7. Провести эксперименты с различными настройками ГА (размер популяции, число поколений, вероятности скрещивания и мутации) и проанализировать их влияние на результат.

В процессе работы были изучены следующие технологии и инструменты:

- язык программирования Python (библиотеки Tkinter для GUI, Matplotlib для построения графиков, NumPy для работы с матрицами);
- принципы проектирования пользовательских интерфейсов и встраивания графиков в приложение;
- методы работы с файловым вводом-выводом для загрузки данных;
- подходы к построению и отладке генетических алгоритмов.

### **1.3 Этапы работы**

1. Формирование бригады, выбор задания и ЯП, распределение ролей в бригаде
2. Демонстрация прототипа GUI и плана по решению задачи (описание формата данных, используемой функции качества, и.т.д.)
3. Промежуточная сдача программы (несколько итераций). На данном этапе должен быть частично-функционирующий GUI, работающий алгоритм, но возможно, требующий корректировок.
4. Демонстрация финальной версии программы и отчета.

### **1.4 Требования к реализации**

- Программа должна иметь GUI.
- Должна быть возможность задать данные через GUI, чтение из файла, случайную генерацию.
- Алгоритмы реализуются самостоятельно без использования готовых решений.
- Настройкой параметров алгоритмов должна производиться пользователем.



- Пошаговая визуализация поиска решения (как меняется аппроксимирующая функция, текущий экстремум, текущее решение оптимизационной задачи в зависимости от популяции). Должен быть выбор переключения между разными текущими решениями.
- Возможность перейти к конечному решению пропустив отображение всех шагов.
- Должен присутствовать график изменения функции качества решения от номера популяции. График дополняется с каждым шагом алгоритма.

## **2 РЕЗУЛЬТАТЫ РАБОТЫ**

### **2.1. Организация работы и распределение подзадач**

В работе участвовала бригада из трёх человек. Для эффективной параллельной разработки программа была разделена на три функциональных модуля, каждый из которых закреплён за отдельным участником. Первому участнику была выдана задача разработки графического интерфейса на основе Tkinter, включая создание окон ввода параметров, отображение матрицы затрат в виде таблицы, визуализацию текущего решения и графика сходимости, а также панель навигации с кнопками управления. Вторым участником отвечает за реализацию ядра генетического алгоритма: функции создания начальной популяции, расчёта целевой функции и приспособленности, отбора методом масштабирования, упорядоченного скрещивания, мутации обменом. Третьему участнику была поручена разработка контроллера — связующего слоя между GUI и ядром алгоритма, который управляет состоянием приложения, хранит историю популяций для возврата на несколько шагов назад, обрабатывает команды пошаговой навигации и обновляет данные на экране. Такое разделение позволило вести разработку независимо, используя общий контракт в виде структур данных и названий функций, а сборка итогового приложения производится путём импорта всех трёх модулей в единую точку входа.

### **2.2. Структура данных и хранение состояния**

В основе программы лежат следующие структуры данных:

Матрица затрат представляет собой двумерный список размером  $N \times N$ , содержащий положительные числа  $c[i][j]$  — затраты на назначение  $j$ -й работы  $i$ -му кандидату.

Хромосома (решение) — список целых чисел длины  $N$ , представляющий перестановку, где индекс соответствует кандидату, а значение — номеру назначенной работы.

Популяция — список хромосом, размер которого задаётся пользователем.

В контроллере ведётся история состояний `history`, содержащая последние три поколения для реализации навигации назад. Каждое состояние включает номер поколения, полную популяцию, список стоимостей, лучшее решение и его стоимость, среднюю стоимость по популяции, число скрещиваний и мутаций, процент улучшения, а также счётчик поколений без улучшения.

Помимо этого, для построения графика используется отдельный список `plot_history`, в котором сохраняются все когда-либо созданные поколения в виде сводных записей – номер поколения, лучшая стоимость, средняя стоимость и лучшая особь. Это позволяет отображать полную историю сходимости, не перегружая память хранением всех популяций.

Глобальное лучшее решение – хромосома с минимальной стоимостью за всё время работы алгоритма – сохраняется отдельно в кортеже `global_best` и обновляется при каждом улучшении.

В контроллере также поддерживаются флаг досрочной остановки `stopped` и счётчик `no_improvement_counter`, которые определяют, следует ли завершить эволюцию при отсутствии улучшения на протяжении заданного числа поколений.

В интерфейсе дополнительно используется таблица, которая отображает все особи текущей популяции с их хромосомами и стоимостями, отсортированными по возрастанию затрат.

### **2.3. Генетический алгоритм. Операторы и параметры.**

Целевая функция — сумма затрат для заданной хромосомы:

$$\text{Fitness} = \sum_{i=0}^{n-1} \text{matrix}[i][\text{chromosome}[i]]$$

Приспособленность — величина, обратная целевой функции (для преобразования задачи минимизации в задачу максимизации при отборе):

$$\text{FitnessScore} = \frac{1}{\text{Fitness} + 1}$$

Метод отбора — метод масштабирования приспособленности: вероятность выбора особи пропорциональна её значению приспособленности.

Скрещивание — упорядоченное. Сохраняет относительный порядок генов родителей, гарантирует отсутствие повторений.

Мутация — обмен двух случайных позиций в хромосоме.

Элитизм — при формировании нового поколения 2% лучших особей из предыдущей популяции переносятся без изменений.

Оставшиеся особи создаются следующим образом:

- выбор двух родителей методом рулетки;
- скрещивание (при отсутствии скрещивания потомки идентичны родителям);
- мутация полученного потомка.
- Остальные параметры (размер популяции, число поколений, вероятности) настраиваются пользователем через GUI.

#### **2.4. Идея синхронизации GUI и ГА**

Контроллер выступает в качестве связи между пользовательским интерфейсом и ядром алгоритма. Его функции:

Инициализация — получение от GUI параметров (N, размер популяции, число поколений, вероятности, матрица затрат), запуск первого поколения через ядро ГА.

Хранение состояния — поддержка текущего индекса поколения, истории популяций, глобального лучшего решения.

Обработка команд навигации:

- «Шаг вперёд» — запуск одной итерации ГА (отбор, скрещивание, мутация, формирование нового поколения), обновление истории, передача обновлённых данных в GUI.
- «Назад» — откат к предыдущему поколению (из истории), перерисовка интерфейса с сохранёнными данными.
- «В конец» — выполнение всех оставшихся поколений без промежуточных остановок, отображение финального результата.

Обновление GUI — после каждого изменения состояния контроллер передаёт в интерфейс:

- текущую популяцию (для отображения лучшего решения в ней);
- глобальное лучшее решение и его стоимость;
- новые точки для графика сходимости;
- статистику (номер поколения, число скрещиваний, число мутаций).

Синхронизация организована так, что ядро ГА не содержит кода визуализации и работает независимо от GUI. Контроллер вызывает функции ядра, получает результаты и передаёт их в интерфейс через заранее определённые методы обновления.

## **2.5. Реализация GUI**

Графический интерфейс реализован в классе App и обеспечивает взаимодействие пользователя с программой. При запуске отображается стартовое окно, содержащее поля для ввода размерности задачи  $N$ , размера популяции, максимального числа поколений, вероятностей скрещивания и мутации, а также переключатели способа ввода матрицы затрат (ручной, из файла или случайная генерация). После ввода параметров и нажатия кнопки «Начать» открывается соответствующее окно: для ручного ввода — таблица размером  $N \times N$  для заполнения значений, для файла — чтение из текстового файла, для случайного — генерация случайных целых чисел от 1 до 100.

Главное окно содержит левую панель с графиком сходимости, текстовым полем статистики и панелью навигации (кнопки «Назад», «Вперёд», «В конец», «Сброс», «Новый запуск»), а правую панель — с таблицей матрицы затрат, полем для ввода номера особи и кнопкой «Показать», текстовым полем для вывода информации о решении, а также таблицей всех решений текущей популяции с сортировкой по стоимости.

Подписи строк и столбцов матрицы выполнены с нулевой нумерацией для соответствия внутреннему представлению данных. На графике одновременно отображаются две кривые: синяя линия с маркерами — лучшая

стоимость в каждом поколении, красная пунктирная – средняя стоимость по популяции, что позволяет оценивать сходимость алгоритма. График поддерживает интерактивность: клик по точке на синей линии вызывает всплывающее окно с информацией о поколении. Пример графического интерфейса представлен в приложении А.

Ключевые методы класса App:

`__init__(self, root)` – конструктор, принимает корневой объект Tkinter, инициализирует поля и вызывает `show_startup()`.

`clear(self)` – удаляет все дочерние виджеты из окна для переключения экранов.

`show_startup(self)` – создаёт стартовое окно с элементами управления, не возвращает значения.

`start(self)` – считывает параметры из полей ввода, в зависимости от выбранного способа ввода вызывает соответствующий метод (`show_matrix_input`, чтение файла или генерация матрицы) и затем `init_controller_and_show_main()`.

`show_matrix_input(self)` – создаёт таблицу для ручного ввода значений матрицы, сохраняет ссылки на поля ввода в `self.entries`.

`confirm_matrix(self)` – считывает значения из таблицы, проверяет корректность, сохраняет матрицу в `self.matrix` и вызывает `init_controller_and_show_main()`.

`setup_plot(self, parent)` – создаёт фигуру Matplotlib, настраивает оси, создаёт две пустые линии – `line_best` (синяя, с маркерами и включённым режимом `picker`) и `line_mean` (красная пунктирная), добавляет легенду, встраивает график в родительский виджет и подключает обработчик события `pick_event` к методу `on_dot`.

`redraw_plot(self)` – полностью перерисовывает график на основе данных из `controller.get_plot_data()`, обновляя обе линии и автоматически масштабируя оси.

`update_stats(self, state)` – принимает словарь состояния, формирует строку статистики, включая поколение, лучшую и среднюю стоимость, лучшую особь, число скрещиваний и мутаций, процент улучшения, глобальное лучшее решение, а также, при остановке алгоритма, добавляет соответствующее сообщение.

`update_nav_buttons(self)` – управляет активностью кнопок навигации на основе текущего индекса, размера истории, достижения максимума и флага остановки.

`update_table(self, state)` – заполняет таблицу `tree` данными текущей популяции: каждая строка содержит исходный индекс особи, хромосому и стоимость; строки сортируются по возрастанию стоимости.

`update_display(self)` – метод обновления интерфейса. Вызывает `controller.get_current_state()`, затем последовательно обновляет график (`redraw_plot`), статистику (`update_stats`), состояние кнопок (`update_nav_buttons`), подсвечивает лучшее решение в матрице (`highlight_solution`), обновляет таблицу решений (`update_table`) и выводит в поле `solution_display` информацию о лучшем решении текущей популяции.

`highlight_solution(self, permutation)` – принимает хромосому-перестановку, сбрасывает цвет всех ячеек на белый и окрашивает в зелёный цвет ячейки, соответствующие назначениям.

`show_solution(self)` – считывает номер из поля `solution_index_var`, проверяет его на корректность, извлекает из текущего состояния популяцию и стоимости, получает особь по индексу и её стоимость, вызывает `highlight_solution` и выводит информацию в `solution_display`.

`_format_solution_text(self, permutation, cost, label)` – вспомогательный метод, формирует строку с информацией о решении (работники и задачи с нумерацией с нуля) и возвращает её.

`show_main(self)` – строит главное окно: создаёт левую панель с графиком, полем статистики, кнопками навигации и информацией о параметрах, а также правую панель с матрицей, элементами управления,

полем вывода и таблицей популяции. В конце вызывает `self.update_display()` для начальной отрисовки.

`on_forward(self)` – обработчик кнопки «Вперёд», вызывает `controller.step_forward()` и затем `update_display()`.

`on_back(self)` – обработчик кнопки «Назад», вызывает `controller.step_back()` и затем `update_display()`.

`on_end(self)` – обработчик кнопки «В конец», в цикле выполняет шаги вперёд, пока не будет достигнут максимум или не сработает критерий остановки, обновляя интерфейс после каждого поколения для визуализации процесса.

`on_reset(self)` – обработчик кнопки «Сброс», вызывает `controller.reset()`, очищает график и вызывает `update_display()`.

`on_restart(self)` – обработчик кнопки «Новый запуск», сбрасывает состояние приложения (контроллер, матрицу, параметры, виджеты) и возвращает к стартовому окну.

`on_dot(self, event)` – обработчик клика по точке на графике: проверяет, что событие относится к линии `line_best`, извлекает индекс точки, получает соответствующие данные из `plot_history` и выводит информационное сообщение.

Все методы, изменяющие состояние контроллера, после выполнения вызывают `update_display()` для синхронизации интерфейса с актуальными данными.

## **2.6. Реализация генетического алгоритма**

Генетический алгоритм (класс `GeneticAlgorithm`) реализует эволюционные операторы. Он получает размерность  $n$ , матрицу затрат, размер популяции, вероятности скрещивания и мутации, размер элитной группы. Хромосома представлена перестановкой целых чисел от 0 до  $n-1$ , где индекс соответствует кандидату, а значение — номеру работы.

Основные методы:



`__init__(self, n, matrix, population_size=50, cross_prob=0.8, mutation_prob=0.05, elite_size=2)` – конструктор, сохраняет параметры.

`initPopulation(self)` – создаёт список из `population_size` случайных перестановок и возвращает его.

`evaluate(self, population)` – принимает популяцию, для каждой хромосомы вычисляет сумму затрат по матрице (`sum(self.matrix[i][chromosome[i]])`), возвращает список стоимостей, минимальную стоимость и соответствующую хромосому.

`scaleFitness(self, costs)` – принимает список стоимостей, вычисляет приспособленность как  $1.0 / \text{cost}$ , затем выполняет линейное масштабирование к диапазону `[0.5, 1.0]` для уменьшения разброса, и возвращает нормализованные вероятности (сумма равна 1). При равенстве всех стоимостей возвращает равномерное распределение.

`rouletteSelect(self, probs)` – принимает список вероятностей, генерирует случайное число и возвращает индекс выбранной особи методом рулетки.

`selectParents(self, costs)` – вызывает `scaleFitness` для получения вероятностей, затем дважды вызывает `rouletteSelect` для выбора двух разных индексов родителей и возвращает их.

`orderCrossover(self, parent1, parent2)` – реализует упорядоченный кроссовер (ОХ): выбирает случайный отрезок в родителях, копирует его в потомков, а оставшиеся позиции заполняет генами из другого родителя в порядке обхода, сохраняя относительный порядок. Возвращает двух потомков.

`swapMutation(self, chromosome)` – выполняет мутацию обменом: выбирает две случайные позиции и меняет их местами, возвращает изменённую хромосому.

`step(self, population, costs)` – основной метод эволюции. Принимает текущую популяцию и её стоимости. Сначала сохраняет элитные особи (первые `elite_size` лучших) в новую популяцию. Затем в цикле, пока размер новой популяции не достигнет `population_size`, выбирает двух родителей через

`selectParents`, с вероятностью `cross_prob` выполняет кроссовер (иначе копирует родителей), затем с вероятностью `mutation_prob` мутирует каждого потомка. Вычисляет стоимости потомков и добавляет их в новую популяцию. По завершении вычисляет процент улучшения как разницу между лучшей стоимостью старой и новой популяции, делённую на старую стоимость, а также среднюю стоимость новой популяции. Возвращает новую популяцию, список её стоимостей и словарь статистики (`num_crossovers`, `num_mutations`, `improvement`, `mean_cost`).

Генетический алгоритм не содержит логики управления или отображения и полностью независим от остальных модулей.

## **2.7. Реализация контроллера**

Контроллер (класс `Controller`) управляет состоянием алгоритма и историей поколений. Он получает параметры задачи и создаёт экземпляр генетического алгоритма. Хранит основную историю состояний `history` в виде списка словарей, ограниченного последними `max_history` поколениями (по умолчанию 3), что позволяет экономить память и одновременно поддерживать навигацию назад.

Для отображения полной истории сходимости используется отдельный список `plot_history`, в котором сохраняются все поколения (номер, лучшая стоимость, средняя стоимость, лучшая особь) без удаления. Каждое состояние в `history` содержит: номер поколения, популяцию, список стоимостей, лучшее решение и его стоимость, среднюю стоимость, число скрещиваний и мутаций, процент улучшения, а также счётчик поколений без улучшения. Контроллер хранит глобальное лучшее решение в кортеже `global_best` и поддерживает флаг `stopped` и счётчик `no_improvement_counter` для реализации критерия досрочной остановки (отсутствие улучшения глобального лучшего в течение 20 поколений).

Методы контроллера:

`__init__(self, n, matrix, pop_size, cross_prob, mut_prob, max_gen, elite_size=2, max_history=3)` – конструктор, сохраняет параметры, создаёт

экземпляр GeneticAlgorithm, инициализирует пустые истории и вызывает `_init_first_generation()`.

`_init_first_generation(self)` – инициализирует начальную популяцию через `ga.initPopulation()`, вычисляет стоимости через `ga.evaluate()`, устанавливает глобальное лучшее решение, создаёт первое состояние с номером поколения 0 и помещает его в `history` и `plot_history`, сбрасывает счётчик и флаг остановки, устанавливает `current_index = 0`.

`get_current_state(self)` – возвращает состояние, на которое указывает `current_index`; используется GUI для получения данных для отображения.

`step_forward(self)` – генерирует новое поколение, если алгоритм не остановлен. Берёт последнее состояние, вызывает `ga.step()` для получения новой популяции и статистики, вычисляет лучшее решение, обновляет глобальное лучшее, при улучшении обнуляет счётчик `no_improvement_counter`, иначе увеличивает его; при достижении 20 устанавливает флаг `stopped`. Формирует новое состояние, добавляет его в `history` и соответствующую запись в `plot_history`. Если размер `history` превышает `max_history`, удаляется самое старое состояние с коррекцией индекса. В конце устанавливает `current_index` на последнее состояние и возвращает его.

`step_back(self)` – если `current_index > 0`, уменьшает индекс на 1, урезает `history` до этого индекса, обрезает `plot_history` до текущего поколения, сбрасывает флаг остановки и восстанавливает счётчик `no_improvement_counter` из состояния, затем возвращает текущее состояние.

`reset(self)` – полностью сбрасывает состояние контроллера, вызывая `_init_first_generation()`, и возвращает начальное состояние.

`is_stopped(self)` – возвращает значение флага `stopped`.

`get_plot_data(self)` – возвращает `plot_history` для построения графика.

Контроллер не содержит кода отображения и не зависит от GUI. Взаимодействие с GUI происходит через методы, возвращающие данные, и через обращение к полям `history`, `current_index`, `global_best`, `stopped`. GUI

самостоятельно вызывает методы контроллера при нажатии кнопок и после этого обновляет интерфейс, используя полученные данные.

## **2.8. Реализация навигации**

Данные организованы в контроллере в виде списка `history`, хранящего состояния поколений. Каждое состояние содержит полную популяцию (список хромосом) и соответствующие стоимости, что позволяет при возврате назад полностью восстановить предыдущее состояние без пересчёта.

Ограничение истории тремя поколениями обеспечивает контроль использования памяти, так как размер популяции и размерность задачи могут быть значительными. Указатель `current_index` определяет текущее отображаемое поколение. При шаге вперёд, если есть сохранённые будущие поколения, просто увеличивается индекс; если достигнут конец истории – генерируется новое поколение. При шаге назад уменьшается индекс и урезаются обе истории – `history` и `plot_history`, что исключает возможность перехода к удалённым будущим поколениям. Это сделано для того, чтобы при повторном шаге вперёд поколения создавались заново, а не брались из «будущего», которое могло бы быть неактуальным после изменений.

Отдельный список `plot_history` позволяет сохранять все поколения для полного отображения на графике, даже после удаления популяций из основной истории. Критерий досрочной остановки (20 поколений без улучшения) автоматически завершает эволюцию, если алгоритм застрял в локальном оптимуме, что экономит время и ресурсы. Кнопка «Сброс» полностью переинициализирует контроллер, очищая обе истории и начиная с нового случайного поколения, а кнопка «Новый запуск» возвращает приложение к стартовому окну для ввода новых параметров.

### 3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

Для реализации программы использованы следующие библиотеки языка Python:

- `tkinter` (включая `ttk` и `messagebox`) – для создания графического интерфейса и стандартных диалоговых окон;
- `matplotlib.pyplot` – для построения графика сходимости;
- `matplotlib.backends.backend_tkagg.FigureCanvasTkAgg` – для встраивания графика в окно Tkinter;
- `random` – для генерации случайных чисел и перемешивания.

Дополнительных внешних библиотек не требовалось. Выбор Tkinter обусловлен его наличием в стандартной версии Python, что обеспечивает переносимость программы без дополнительной установки. Matplotlib выбран благодаря удобству работы с графиками и возможности интеграции с Tkinter.

## 4 ОТЗЫВ РУКОВОДИТЕЛЯ

Если прохождение практики было по кафедральным темам достаточно добавить формулировку:

По результатам работы учебная практика оценена на <ваша\_оценка>.

Если прохождение практики было по своей теме, то необходимо добавить скан отзыва руководителя с оценкой и подписью, скан должен быть в высоком качестве на всю страницу.

Для прохождения автоматической проверки для скана необходима подпись и ссылка в тексте, т.к. он считается рисунком.

Отзыв руководителя с оценкой (см. рис. X).

### Отзыв

#### о прохождении учебной практики

<ФИО студента>, студент(ка) группы <номер группы>, второго курса бакалавриата Санкт-Петербургского электротехнического университета “ЛЭТИ” им В.И. Ульянова, проходил(а) учебную практику по теме “<тема практики>” с <дата начала> по <дата окончания>.

В течение практики обучающийся(аяся) <ФИО студента> выполнял <задание на практику>.

В результате практики обучающийся(аяся) <результат практики>. Получаемую работу обучающийся(аяся) <ФИО студента> выполнял <характеристика личных качеств практиканта>.

По результатам работы <ФИО студента> учебной практику оцениваю на <оценка: Отлично, Хорошо, Удовлетворительно, Неудовлетворительно>.

Подпись руководителя практики:

<Должность><дата>

<подпись>/<ФИО>

Рисунок X — Отзыв руководителя

## **ЗАКЛЮЧЕНИЕ**

Работа выполнялась бригадой из трёх человек: К.А. Берсенов, А.С. Большакова, Д.Р. Валитов.

На начальном этапе все участники совместно обсуждали общую архитектуру системы, выбирали структуры данных для хранения популяций и состояний, а также определяли перечень параметров и элементов, выводимых в интерфейс.

Распределение ролей: первый участник - К.А. Берсенов - отвечал за разработку графического интерфейса пользователя – реализовал все окна, таблицы, график и методы обновления отображения. Второй участник - Д.Р. Валитов - разрабатывал ядро генетического алгоритма – реализовал все операторы эволюции и целевые функции. Третья участница - А.С. Большакова - разрабатывала класс контроллера, обеспечивающий управление историей и навигацией, а также реализовывала функции в классе GUI для взаимодействия с контроллером. Кроме того, занималась оформлением формальной части отчёта: аннотации, постановки задач, описания предметной области и выводов, а также корректировкой основных разделов. Описание отдельных модулей программы и реализованных в нем функций каждый участник составлял самостоятельно. После каждой итерации участники сообща определяли логику исправления ошибок, и каждый реализовывал поправки к своему модулю.

## **СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ**

1. Панченко, Т. В. Генетические алгоритмы: учебно-методическое пособие/ под ред. Ю. Ю. Тарасевича. - Астрахань: Издательский дом «Астраханский университет», 2007.
2. Вирсански Э., Генетические алгоритмы на Python / пер. с англ. А. А. Слинкина. – М.: ДМК Пресс, 2020. – 286 с.



## ПРИЛОЖЕНИЕ А

Первое окно ввода параметров (см. рис. 1)

Задача о назначениях

N: 6

Размер популяции: 50

Макс. поколений: 100

Вер. скрещивания: 0.8

Вер. мутации: 0.05

Способ ввода: ☒ Ручной ☐ Файл ☐ Случайный

Начать

Рисунок 1 – Окно ввода параметров.

Второе окно ввода матрицы (см. рис. 2)

Задача о назначениях

Матрица 6x6

Задачи

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
|   | 0 | 1 | 2 | 3 | 4 | 5 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| 2 | 0 | 0 | 0 | 0 | 0 | 0 |
| 3 | 0 | 0 | 0 | 0 | 0 | 0 |
| 4 | 0 | 0 | 0 | 0 | 0 | 0 |
| 5 | 0 | 0 | 0 | 0 | 0 | 0 |

OK Назад

Рисунок 2 – Окно ввода матрицы.

## Стартовое окно решения с нулевым поколением (см. рис. 3)

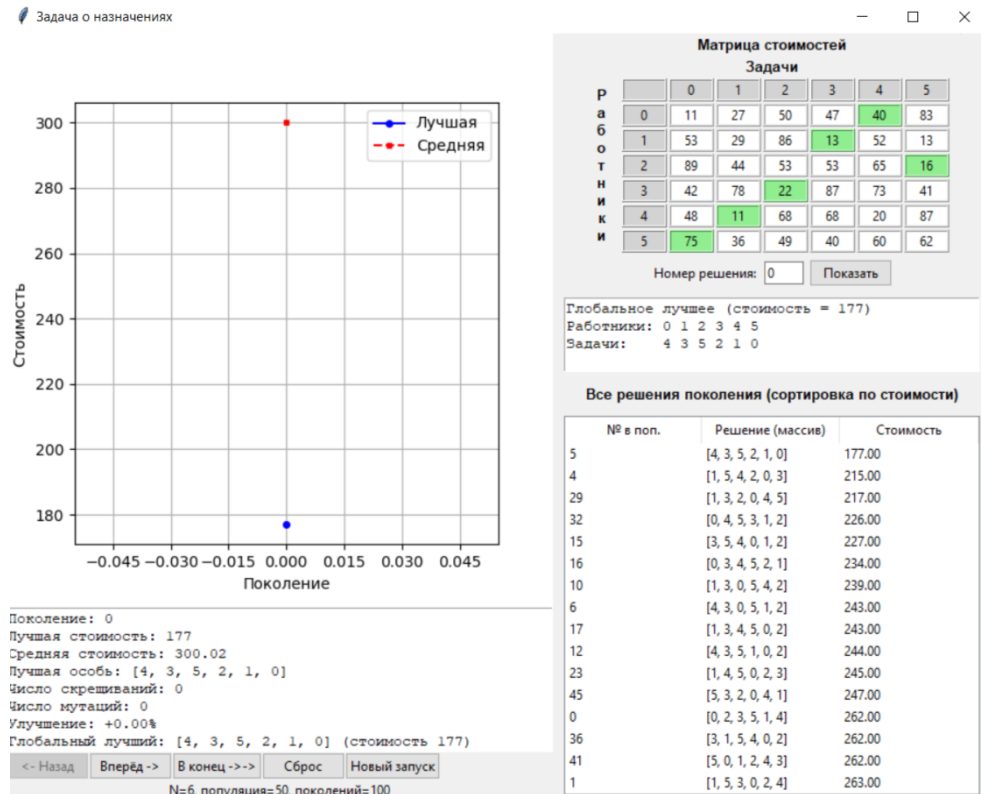


Рисунок 3 – Стартовое окно решения.

## Окно при пошаговом решении (см. рис. 4)

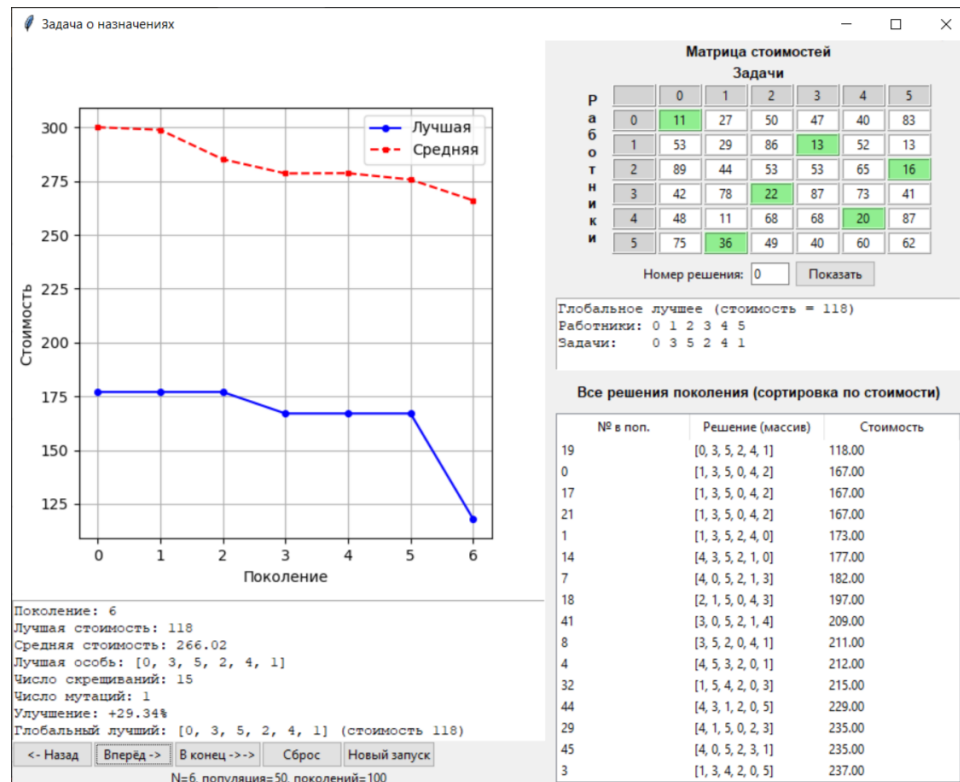


Рисунок 4 – Окно при пошаговом решении.

## Конечный вариант решения (см. рис. 5)

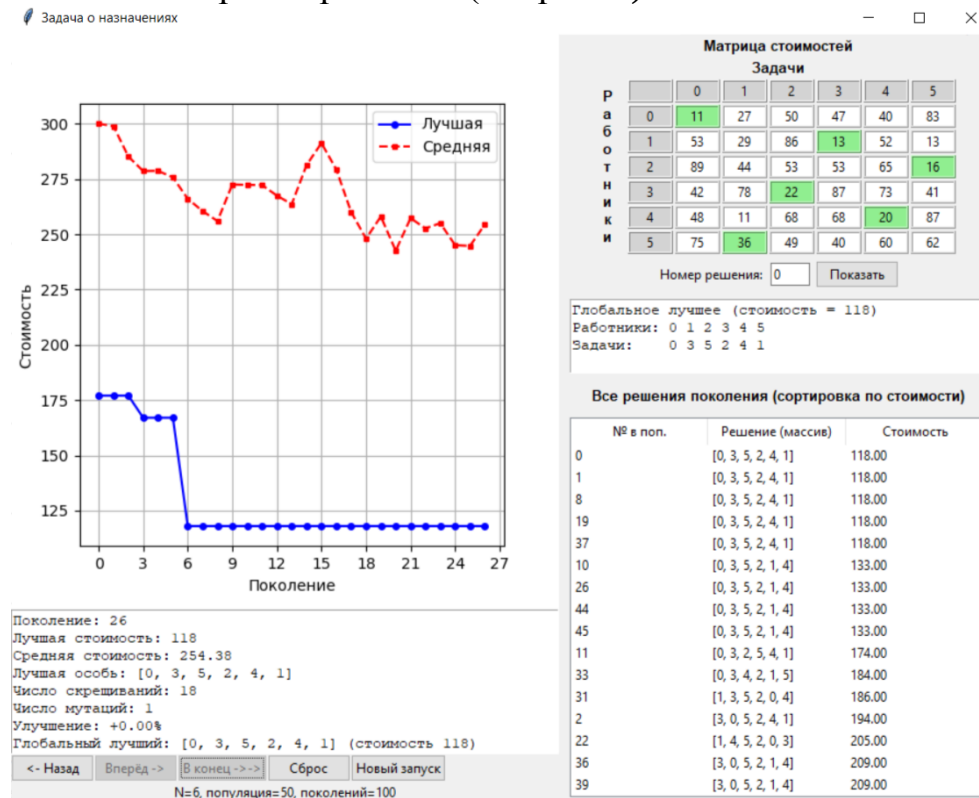


Рисунок 5 – Конечный вариант решения.